Chapter 18 worked solutions

Language Understanding

This guide covers the written solutions to Exercises 1–4 and 9–11. The companion Python notebook covers Exercises 5–8.

1. Find what word counts lose

Choose the vocabulary order *(Ari, called, Sam)*. Each coordinate counts occurrences of the corresponding word, wherever it appears in the sentence.

Sentence	Ari	called	Sam	Count vector
Ari called Sam	1	1	1	$(1, 1, 1)$
Sam called Ari	1	1	1	$(1, 1, 1)$

Both sentences contain the same words in the same quantities. A classifier that receives only these counts gets the same input for both sentences and has no evidence about who made the call.

In these sentences, the name before *called* is the caller and the name after it is the person receiving the call. A Transformer encoder with positional information can use self-attention to relate each name to the verb and distinguish the two positions. The output vector for *Ari* can then reflect that Ari is the caller in the first sentence and the recipient in the second.

2. Pool only the valid rows

The mask $(1, 1, 1, 0)$ selects the first three rows. The fourth row contributes neither a feature vector to the sum nor a row to the denominator.

$$\begin{aligned} \text{Masked mean} &= \frac{(2,0)+(1,3)+(0,1)}{3} = \frac{(3,4)}{3} = \left(1, \frac{4}{3}\right), \\ \text{Unmasked mean} &= \frac{(2,0)+(1,3)+(0,1)+(6,6)}{4} = \frac{(9,10)}{4} = \left(\frac{9}{4}, \frac{5}{2}\right). \end{aligned}$$

The padding row $(6, 6)$ pulls the unmasked mean upward in both coordinates. To see when a new row leaves an unmasked mean unchanged, let $\mathbf{m}$ be the mean of the n rows already included, with $n > 0$, and let $\mathbf{p}$ be the padding vector. The old sum is $n\mathbf{m}$. For the new mean to equal $\mathbf{m}$, we need

$$\frac{n\mathbf{m} + \mathbf{p}}{n+1} = \mathbf{m} \quad \Leftrightarrow \quad \mathbf{p} = \mathbf{m}.$$

Thus the added row must equal the old mean in every coordinate. If the first three rows are the only ones present, that would require a padding row of $(1, 4/3)$. Masked pooling gives the same result for any padding vector as long as its mask entry stays zero.

3. Assign entity tags

At the word level, the tags for *Sam left Lake City* are `B-PER`, `O`, `B-LOC`, and `I-LOC`. In IOB2, `B` begins an entity, `I` continues an entity of the same type, and `O` marks a word outside all entities. *Sam* begins a one-word person entity. *Lake* begins the location and *City* continues it.

For an illustrative split into `La` and `##ke`, give both pieces the word ID for *Lake*. The beginning special token has no word ID, which the companion code represents as -1.

Token	Word ID	Supervised tag	Target ID
`[CLS]`	-1	Ignored	-100
`Sam`	0	`B-PER`	1
`left`	1	`O`	0
`La`	2	`B-LOC`	2
`##ke`	2	Ignored	-100
`City`	3	`I-LOC`	3

The first piece of each word receives the word's tag. The continuation piece `##ke` and `[CLS]` have zero weight in the tagging loss. `O` is a supervised label for a real word such as *left*. At evaluation time, take predictions at `Sam`, `left`, `La`, and `City` to recover one predicted tag per word.

4. Compare tag and span accuracy

Write each entity as a triple *(start, end, type)* with zero-based word indices. The start is included and the end is excluded. For example, $(0, 2, ORG)$ covers words 0 and 1.

Tag sequence	Typed spans
Gold	$(0, 2, ORG)$, $(3, 4, PER)$
Predicted	$(0, 2, ORG)$, $(3, 4, LOC)$

The tag sequences agree at indices 0, 1, 2, and 4. They disagree at index 3, where the prediction is `B-LOC` and the gold tag is `B-PER`. Word accuracy is therefore $4/5 = 80\%$.

Only $(0, 2, ORG)$ matches exactly. Precision divides the match count by the two predicted entities. Recall divides it by the two gold entities.

$$\text{Exact span precision} = \frac{1}{2} = 50\%, \qquad \text{Exact span recall} = \frac{1}{2} = 50\%.$$

If we matched only start and end, both spans would count as matches and both scores would reach 100%. The single-word entity at $[3, 4)$ has the correct boundaries but the tagger assigned the location type to a person.

9. Change the label mix (optional)

Take *update* as the positive class. Each office receives 200 messages, so the first office has $200 - 60 = 140$ lookups and the second has $200 - 20 = 180$. The model flags 90% of each office's actual updates and 5% of each office's actual lookups. Apply each rate to the corresponding group to get the expected counts.

Expected count	Office 1	Office 2
Updates flagged (true positives)	$0.90 \times 60 = 54$	$0.90 \times 20 = 18$
Updates missed (false negatives)	$60 - 54 = 6$	$20 - 18 = 2$
Lookups flagged (false positives)	$0.05 \times 140 = 7$	$0.05 \times 180 = 9$
Lookups correctly identified (true negatives)	$140 - 7 = 133$	$180 - 9 = 171$
Total messages flagged	$54 + 7 = 61$	$18 + 9 = 27$

Precision is the fraction of flagged messages that are actual updates. Using the expected counts gives

$$\text{Office 1 precision} = \frac{54}{61} \approx 88.5\%, \qquad \text{Office 2 precision} = \frac{18}{27} \approx 66.7\%.$$

The second office has fewer updates available to find and more lookups that could trigger false alarms. The same recall and false-positive rate therefore give lower precision there.

Check whether recall on indirectly phrased updates differs between offices. For example, label messages such as *I moved last week* using the same annotation rule in both offices. Apply the same decision threshold, then report how many indirect updates each office has and what fraction the classifier flags. The office-wide recall in the exercise could be the same even if the classifier handles indirect requests differently.

10. Plan an independent split (optional)

Imagine three offices that all use the template *Please send a copy of the address on file*. A customer writes to Office A and follows up with Office B. A model could score well on a random message split by recognizing the repeated request or remembering that customer's earlier message. The unit held out should match the question we want the test to answer.

To test new customers, group by stable customer ID across all offices. Assign an entire customer, including every thread and office they used, to training, validation, or test. A customer in the test set must be absent from both training and validation. Keep several offices represented in the test if the claim is about new customers at familiar offices. Report the number of customers and messages, and show the label counts and office mix for each partition.

To test a new office, reserve one office's messages for test and select models using other offices. Suppose Office C is held out and a customer who used C also used Office A. Remove that customer's A messages from training and validation and record the exclusion. This makes the primary test a new-office comparison without a familiar-customer shortcut. For a comparison with several held-out offices, repeat the procedure separately for each office and report the results by office. This is a different experiment from drawing new customers within a known office.

Audit templates as well as IDs. First compare exact normalized text after replacing variable slots such as names and addresses with placeholders. For example, *Update Ana's address to 14 Oak Road* and *Update Ben's address to 90 Elm Road* can share the same template. Next flag likely paraphrases using word or character overlap and semantic similarity, then inspect examples near the proposed match boundary by hand. Common topic alone does not make two requests equivalent. *Please confirm my new address* and *Please enter my new address* may express different intents. Save the resulting template-family IDs and the rule that created them.

For a test that also claims to generalize to unfamiliar wording, keep each template family in one partition. One way is to connect records that share a customer, thread, or template family and assign whole connected groups. For the new-office test, keep the held-out office intact and remove from training and validation any linked customer or prohibited template family. Recheck both customer and template overlap after these removals. Report how many examples were dropped, since strict grouping can make common templates hard to evaluate separately. Distinguish performance on familiar office templates from performance on unfamiliar wording if both are relevant to the intended use.

11. Design an entity error study (optional)

Suppose the test contains *Lake City*. The model can tag the whole name correctly, predict only *Lake*, or identify the full two-word phrase with the wrong type. Those outcomes answer different questions. Evaluate both taggers on the same documents and keep the original word boundaries and gold location spans fixed.

Build the **seen** list from full gold location names in the training split. Normalize the surface forms by a predeclared rule, such as case folding and collapsing runs of spaces. Apply the same rule to complete gold names in the test set. A test name is seen when its full normalized string occurs in the training location list. It is unseen otherwise. This definition measures exposure in the task training data. A pretrained tagger may have encountered an unseen name elsewhere.

Score a prediction as an exact match when its start word, exclusive end word, and type all match the gold span. For diagnosis, compare unmatched predicted spans with unmatched gold spans in the same sentence. Match overlapping spans one to one using a fixed rule, such as greatest shared word count with a deterministic tie break. An overlapping span of the correct type with a different start or end is a **boundary error**. A span with the right boundaries and the wrong type is a **type error**. Record cases with both differences separately. A gold span with no matched prediction is a miss. A prediction with no matched gold span is a false positive. Use strict IOB2 decoding or a common declared repair policy for both systems. The diagnostic pairing does not turn a boundary or type error into a correct span.

Assign these outcomes to the slice of the **full gold span**. If the gold name *Lake City* is unseen and a predicted location covers only *Lake*, count a boundary error in the unseen slice. Checking *Lake* against the seen list instead could move that error into the wrong group. Report unpaired false positives separately because they have no gold name that defines a slice.

For each slice, report its support, the number of gold mentions, and exact span recall, the number of correctly recovered gold mentions divided by that support. Report the boundary, type, combined-error, and miss counts alongside recall for each tagger. Also report overall exact location precision. Divide the number of exactly matched location spans by the number of predicted location spans, including spurious and wrong-boundary locations. If a slice is small, include the raw numerator and denominator so one extra success is visible. Report paired differences on the same documents and resample documents, rather than individual mentions, if computing uncertainty.

Finally, record whether at least one word in each gold name splits into multiple subwords. Compare recall and error counts for split and unsplit names within each seen and unseen slice. Use the same tokenizer to define this diagnostic for both taggers, or report each tokenizer's fragmentation separately. Convert token predictions back to original word indices with the declared alignment rule before scoring. In a first-piece policy, special tokens and later pieces do not create extra word predictions. Inspect a few multi-piece errors to check whether they come from alignment, from losing an entity boundary, or from choosing the wrong type.